# AI calcs

same colour = same number · yellow = worked out · red = careful

## L2–3 SEARCH

### A* vs UCS vs Greedy (which node next)

**P: g 120, h 200  Q: g 90, h 250  R: g 150, h 160**

$f = \underset{g}{\text{cost so far}} + \underset{h}{\text{guess to goal}}$

A*: P 120+200 = 320, Q 90+250 = 340, R 150+160 = 310
→ smallest f → **R (310)**

UCS: smallest g only → **Q (90)**  g = add up road costs so far

Greedy: smallest h only → **R (160)**

A* is optimal only if h never overestimates (admissible)

### Simulated annealing accept prob

**ΔE = 3 (worse by 3)  T = 2**

$P = e^{-\Delta E / T} = e^{-3/2} = e^{-1.5}$

$= 0.2231$

better move → always accept. high T = accepts more bad moves

### GA roulette wheel selection

**fitness 24  30  16  8**

$P(\text{pick}) = \frac{\text{fitness}}{\text{sum of all fitness}}$

sum = 24 + 30 + 16 + 8 = 78

$P = \frac{24}{78} = 0.3077$

### N-queens successors

successors = $n$ queens × ($n$ − 1) other rows

$= 8 \times 7 = 56$

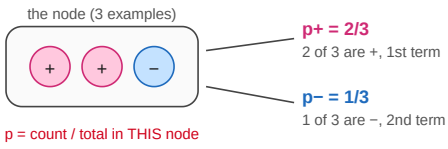
4-queens → 4 × 3 = **12**

each queen can go to every row except the one it is on

## L4 LEARNING FROM EXAMPLES

### Entropy & Gini of a node

**2 +  1 −**




p = count / total in THIS node

$H = -p_+ \log_2 p_+ - p_- \log_2 p_-$

$= -\frac{2}{3}\log_2\frac{2}{3} - \frac{1}{3}\log_2\frac{1}{3}$

$= 0.39 + 0.5283 = 0.9183$

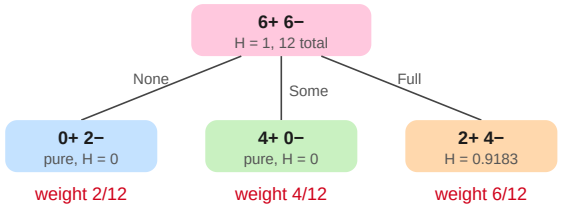
$Gini = 1 - (p_+^2 + p_-^2)$

$= 1 - (\frac{4}{9} + \frac{1}{9}) = 0.4444$

pure node → 0. 50/50 → H = 1, Gini = 0.5

### Information gain (restaurant, Patrons)

**root 6+ 6−  None 0+ 2−  Some 4+ 0−  Full 2+ 4−**




$IG = H(\text{root}) - \Sigma \frac{\text{branch size}}{\text{total}} H(\text{branch})$

H(root) = 1  None = 0, Some = 0 pure

H(Full) = H(2/6, 4/6) = 0.9183

$IG = 1 - [\frac{2}{12}(0) + \frac{4}{12}(0) + \frac{6}{12}(0.9183)]$

$= 1 - 0.4591 = 0.5409$

Gini: root 0.5, Full 0.4444 → $0.5 - \frac{6}{12}(0.4444) = 0.2778$

pick the attribute with the BIGGEST gain. every branch 50/50 (Type) → IG = 0

### Sum of squared errors

**h(x) = 1 + 2x  (1,4) (2,4) (3,8)**

$SSE = \Sigma (\underset{y}{\text{actual}} - \underset{h(x)}{\text{predicted}})^2$

h(1) = 3, h(2) = 5, h(3) = 7

$= (4-3)^2 + (4-5)^2 + (8-7)^2$

$= 1 + 1 + 1 = 3$

MSE → divide by n (= 1). some slides use ½ SSE

### KNN classify (k = 3)

**query (2,3)  A (1,1)  B (3,4)  B (4,3)  A (0,5)  A (6,6)**

$d = \sqrt{(x_1-x_2)^2 + (y_1-y_2)^2}$

A(1,1): √(1+4) = 2.24  B(3,4): √(1+1) = 1.41

B(4,3): √(4+0) = 2  A(0,5): √(4+4) = 2.83

A(6,6): √(16+9) = 5

3 closest: B, B, A → **B**

no need to square root if you only compare

### Precision, recall, F1

**TP 40  FP 10  FN 20  TN 30**

$acc = \frac{TP + TN}{all}$  $P = \frac{TP}{TP + FP}$  $R = \frac{TP}{TP + FN}$

$acc = \frac{40 + 30}{100} = 0.7$

$P = \frac{40}{40 + 10} = 0.8$  $R = \frac{40}{40 + 20} = 0.667$

$F1 = \frac{2PR}{P + R}$

$= \frac{2(0.8)(0.667)}{0.8 + 0.667} = 0.7273$

P → of the ones I said yes. R → of the real yes

# L5 NEURAL NETS

## Perceptron output + update

x = (1, 1)  w = (0.3, 0.2)  b = −0.4  target y = 0  η = 0.1

out = step( $w \cdot x + b$ )

= 0.3(1) + 0.2(1) − 0.4 = 0.1 ≥ 0 → out = 1

$w_1$ new = $w1 + \eta (y - \text{out}) x1$

= 0.3 + 0.1(0 − 1)(1) = **0.2**

y = target, out is 0 or 1 (not 0.1). right answer → no change

## Forward pass (ReLU then sigmoid)

x1 = 0  x2 = 2  w1,3 = −0.2  w2,3 = −0.4  w3,5 = 1.3

$z_3 = x1\,w1,3 + x2\,w2,3$

= 0(−0.2) + 2(−0.4) = −0.8

$a_3$ = ReLU(−0.8) = 0

$z_5 = 0 \times 1.3 = 0 \rightarrow \sigma(0) = \frac{1}{1+e^{0}} = \mathbf{0.5}$

ReLU: negative → 0. $\sigma(z) = 1 / (1 + e^{-z})$

## Activations & loss (quick ones)

$\sigma(z) = \frac{1}{1+e^{-z}}$  ReLU(z) = max(0, z)  tanh(0) = 0

$\sigma(0) = \frac{1}{1+1} = \mathbf{0.5}$  ReLU(−3) = **0**

loss = $(y - \hat{y})^2$

= $(1 - 0.8)^2 = 0.2^2 =$ **0.04**

square AFTER subtracting

# L6 REINFORCEMENT LEARNING

## Discounted return

rewards −1, −1, +10  γ = 0.5

$G = r_1 + \gamma r_2 + \gamma^2 r_3$

= −1 + 0.5(−1) + 0.25(10)

= −1 − 0.5 + 2.5 = **1**

first reward is NOT discounted

## Q-value with slippery moves

0.8 intended  0.1 / 0.1 sideways  R = −0.04  γ = 0.5

V: 0.39, 0.25, 0.04

$Q = \Sigma P(R + \gamma V(s'))$

= 0.8(−0.04 + 0.5×0.39) + 0.1(−0.04 + 0.5×0.25) + 0.1(−0.04 + 0.5×0.04)

= 0.8(0.155) + 0.1(0.085) + 0.1(−0.02)

= 0.124 + 0.0085 − 0.002 = **0.1305**

bump into wall → you stay, use your own V

## Pick the best action (policy)

A = 0.32  B = 0.44  C = 0.21

value(move) = 0.8 × target + 0.1 × left side + 0.1 × right side

Up = 0.8(0.32) + 0.1(0.44) + 0.1(0.21) = 0.321

Right = 0.8(0.44) + 0.1(0.32) + 0.1(0.21) = 0.405

biggest → **Right**

policy = argmax, so give the ACTION not the number

## TD & Q-learning

V(s) = 0.52  V(s') = 0.21  α = 0.2  γ = 0.9  r = 0

$V(s) \leftarrow V(s) + \alpha (r + \gamma V(s') - V(s))$

= 0.52 + 0.2(0 + 0.9×0.21 − 0.52)

= 0.52 + 0.2(−0.331) = **0.4538**

Q(s,a) = 0.97  max Q(s') = 1.63

$Q \leftarrow Q + \alpha(r + \gamma \max Q(s',a') - Q)$

= 0.97 + 0.2(0 + 0.9×1.63 − 0.97)

= 0.97 + 0.2(0.497) = **1.0694**

Q-learning uses the MAX of next state. SARSA uses the action taken. U(s) = V(s)

## Expected utility (MEU)

$EU = \Sigma P \times U$

A = 0.8(10) + 0.2(−10) = 8 − 2 = 6

B = 1(7) = 7

7 > 6 → **B**

MEU = biggest expected value, even if A can win more

## Direct utility estimate

U(s) = average of reward-to-go seen from s

= $\frac{0.84 + 0.92}{2} =$ **0.88**

reward-to-go = sum of rewards from that state to the end

# L7 CNN

## Conv output size & parameter count

input 7×7×16  32 filters  filter 5×5  stride 2  padding 2

$\text{output} = \frac{\text{input} + 2\,\text{padding} - \text{filter size}}{\text{stride}} + 1$

= $\frac{7 + 4 - 5}{2} + 1 = 3 + 1 = 4 \rightarrow$ **4 × 4 × 32**

params = $(\text{filter} \times \text{filter}) \times \text{channel} \times \text{conv layer}$

= 5×5×16×32 = **12 800**

+ bias → + 1 per filter = 12 800 + 32 = 12 832

channel = depth of the INPUT (RGB = 3). depth of output = no. of filters

## Conv, one output value

| | | |
|---|---|---|
| 3 | 1 | 0 |
| 2 | 0 | 1 |
| 1 | 4 | 1 |

∗

| | | |
|---|---|---|
| 1 | 0 | −1 |
| 1 | 0 | −1 |
| 1 | 0 | −1 |

value = sum of (patch × kernel) + bias

= (3−0) + (2−1) + (1−1) + 0

= 3 + 1 + 0 = **4**

middle column × 0 drops out. slide by stride for the next one

## Max & average pooling (2×2, stride 2)

| | | | |
|---|---|---|---|
| 5 | 9 | 2 | 3 |
| 1 | 9 | 3 | 0 |
| 9 | 6 | 5 | 6 |
| 4 | 6 | 6 | 0 |

max: [5 9 1 9] = 9  [2 3 3 0] = 3

[9 6 4 6] = 9  [5 6 6 0] = **6**

avg (bottom right): $\frac{5 + 6 + 6 + 0}{4} =$ **4.25**

split into blocks, one number per block → 4×4 becomes 2×2

### Receptive field

**conv 3** **conv 3** **pool 2, stride 2** **conv 3**

**r = r + (k − 1) × jump** **jump ×= stride**

start r = 1, jump = 1
**conv 3**: 1 + 2(1) = 3
**conv 3**: 3 + 2(1) = 5
**pool 2 s2**: 5 + 1(1) = 6 → jump = 2
**conv 3**: 6 + 2(2) = **10**

jump only changes AFTER the strided layer

---

### IoU & NMS

**box A (0,0)–(6,4)** **box B (2,0)–(8,4)** **threshold 0.5**

$$\text{IoU} = \frac{\text{overlap}}{\text{area A + area B − overlap}}$$

overlap = (6 − 2)(4) = 16, each box = 24

$$\text{IoU} = \frac{16}{24 + 24 - 16} = \frac{16}{32} = \mathbf{0.5}$$

≥ 0.5 → same object → drop the lower score box

NMS: keep top score, delete boxes overlapping it, repeat

---

## L8 RNN, ATTENTION, NLP

### Simple RNN hidden state update

**x = 0** **w = 0.5** **$h_{t-1}$ = 0.1** **u = 0.75** **b = 0.1**

$$h_t = \tanh(\overset{0.5}{w}\overset{0}{x} + \overset{0.75}{u}\overset{0.1}{h_{t-1}} + \overset{0.1}{b})$$

= tanh( 0.5(0) + 0.75(0.1) + 0.1 )
= tanh( 0.175 )
= **0.1732**

u = recurrent weight (on old h). small input → tanh(z) ≈ z

---

### Softmax attention weights

**score 1 = 2** **score 2 = 1**

$$\alpha_1 = \frac{e^{\text{score 1}}}{e^{\text{score 1}} + e^{\text{score 2}}}$$

$$= \frac{e^{2}}{e^{2} + e^{1}} = \frac{7.3891}{7.3891 + 2.7183}$$

$$= \frac{7.3891}{10.1073} = \mathbf{0.7311}$$

$\alpha_2$ = 1 − 0.7311 = 0.2689

equal scores → equal weights (2, 2 → 0.5 each)

---

### Attention output (weighted sum)

**α = 0.7, 0.3** **v1 = [0, 2]** **v2 = [2, 0]**

$$\text{out} = \overset{0.7}{\alpha_1}\,v_1 + \overset{0.3}{\alpha_2}\,v_2$$

= 0.7[0, 2] + 0.3[2, 0]
= [0, 1.4] + [0.6, 0] = **[0.6, 1.4]**

weights must add to 1 (from softmax)

---

### Bigram probability

**count("the cat") = 3** **count("the") = 5**

$$P(\text{cat} \mid \text{the}) = \frac{\overset{3}{\text{count(the cat)}}}{\overset{5}{\text{count(the)}}}$$

$$= \frac{3}{5} = \mathbf{0.6}$$

sentence prob → multiply each bigram

---

## L10 LOGIC

### Logic, evaluate a sentence

**P = T** **Q = F** **R = F**

**¬P ∨ Q ∧ R ⇒ R**

order: ¬ first, then ∧, then ∨, then ⇒

( ¬T ) ∨ ( F ∧ F ) ⇒ F
= ( F ∨ F ) ⇒ F
= F ⇒ F = **True**

⇒ is only false when T ⇒ F

---

## L11 PROBABILITY & BAYES NETS

### Probability rules

**P(A) = 0.3** **P(B) = 0.5** **P(A∧B) = 0.15**

$$P(A \vee B) = \overset{0.3}{P(A)} + \overset{0.5}{P(B)} - \overset{0.15}{P(A \wedge B)}$$

= 0.3 + 0.5 − 0.15 = **0.65**

**P(A∧B) = 0.12** **P(B) = 0.4**

$$P(A|B) = \frac{\overset{0.12}{P(A \wedge B)}}{\overset{0.4}{P(B)}}$$

$$= \frac{0.12}{0.4} = \mathbf{0.3}$$

P(¬A) → 1 − P(A)

---

### Joint table, P(a|b)

| | b | ¬b |
|---|---|---|
| a | **0.132** | 0.418 |
| ¬a | **0.25** | 0.2 |

$$P(a|b) = \frac{P(a, b)}{P(b)}$$

P(b) = P(a,b) + P(¬a,b) = **0.132** + **0.25** = 0.382 ← a cancels out

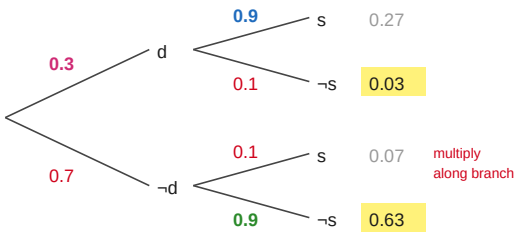
$$P(a|b) = \frac{0.132}{0.382} = \mathbf{0.3455}$$

to get P(b) add the whole b column

---

### Bayes with a negative result (tree)

**P(d) = 0.3** **P(s|d) = 0.9** **P(¬s|¬d) = 0.9**




$$P(d|\neg s) = \frac{P(\neg s|d)\,P(d)}{P(\neg s)}$$

P(¬s) = (1−0.9)(0.3) + 0.9(1−0.3)
= 0.03 + 0.63 = 0.66

$$P(d|\neg s) = \frac{0.03}{0.66} = \mathbf{0.0455}$$

negative result → use the ¬s branches. 1 − P to flip

---

## Disease test (Bayes)

P(D) = 0.01  P(+|D) = 0.9  P(+|¬D) = 0.05

$$P(D|+) = \frac{P(+|D)\ P(D)}{P(+)}$$

P(+) = 0.9×0.01 + 0.05×(1−0.01)
= 0.009 + 0.0495 = 0.0585

$$P(D|+) = \frac{0.009}{0.0585} = \mathbf{0.1538}$$

false alarm rate uses ¬D, which is 0.99 of people

## Which machine made the defect

M1 50%, 2% bad  M2 30%, 3% bad  M3 20%, 5% bad

**P(D) = sum of P(M) P(D|M)**

= 0.5×0.02 + 0.3×0.03 + 0.2×0.05
= 0.01 + 0.009 + 0.01 = 0.029

$$P(M3|D) = \frac{0.01}{0.029} = \mathbf{0.3448}$$

## Bayes net, one full assignment

**P(j, ¬m, ¬a, ¬b, ¬e) = P(¬b) P(¬e) P(¬a|¬b,¬e) P(j|¬a) P(¬m|¬a)**

each node given its parents only

= 0.999 × 0.998 × 0.999 × 0.05 × 0.99
= **0.0493**

table gives P(a|...) → P(¬a|...) = 1 − that

## Bayes net enumeration P(B | j, m)

P(b) = 0.01  P(e) = 0.02  P(a|b,e) .95 .94 .3 .05  P(j|a) .9 / .05
P(m|a) .6 / .02

$$P(b|j,m) = \alpha\ \overset{0.01}{P(b)} \Sigma_e \overset{0.02}{P(e)} \Sigma_a P(a|b,e)\ P(j|a)\ P(m|a)$$

1) j, m part for a and ¬a  2) inside bracket for e and ¬e  3) S(b), S(¬b)  4) divide

**P(j|a) P(m|a)**

= .9 × .6 = .54

**P(j|¬a) P(m|¬a)**

= .05 × .02 = .001

**S(b)**

**[b, e] = P(a|b,e)(.54) + P(¬a|b,e)(.001)**
= .95(.54) + .05(.001) = 0.51305  ¬a = 1 − .95

**[b, ¬e] = P(a|b,¬e)(.54) + P(¬a|b,¬e)(.001)**
= .94(.54) + .06(.001) = 0.50766

**S(b) = P(b) [ P(e) [b, e] + P(¬e) [b, ¬e] ]**
= .01 [ .02(0.51305) + .98(0.50766) ]
= .01 [ 0.010261 + 0.497507 ] = 0.005078

**S(¬b)**

**[¬b, e] = P(a|¬b,e)(.54) + P(¬a|¬b,e)(.001)**
= .3(.54) + .7(.001) = 0.1627

**[¬b, ¬e] = P(a|¬b,¬e)(.54) + P(¬a|¬b,¬e)(.001)**
= .05(.54) + .95(.001) = 0.02795

**S(¬b) = P(¬b) [ P(e) [¬b, e] + P(¬e) [¬b, ¬e] ]**
= .99 [ .02(0.1627) + .98(0.02795) ]  P(¬b) = 1 − .01
= .99 [ 0.003254 + 0.027391 ] = 0.030339

$$P(b|j,m) = \frac{S(b)}{S(b) + S(\neg b)}$$

$$= \frac{0.005078}{0.005078 + 0.030339} = \mathbf{0.1434}$$

do b and ¬b the same way, then divide by the total (that is α)

## Naive Bayes (text)

"great" in + : 8  words in + : 25  vocab : 10  "great" in − : 0
words in − : 14  P(+) = P(−) = 0.5

$$P(great|+) = \frac{\overset{8}{\text{word appears}} + 1}{\underset{25}{\text{words in +}} + \underset{10}{\text{vocab}}}$$

$$= \frac{8 + 1}{25 + 10} = \frac{9}{35} = 0.2571$$

$$P(great|-) = \frac{0 + 1}{14 + 10} = \frac{1}{24} = 0.04167$$

+ : 0.5 × 0.2571 = 0.12857  − : 0.5 × 0.04167 = 0.02083

$$P(+|text) = \frac{0.12857}{0.12857 + 0.02083} = \mathbf{0.8606}$$

+1 so a zero count does not kill the whole product

## Marginalise from a full joint table

| X | Y | Z | P |
|---|---|---|---|
| x1 | y1 | z1 | .06 |
| x1 | y1 | z2 | .09 |
| x1 | y2 | z1 | .14 |
| x1 | y2 | z2 | .11 |
| x2 | y1 | z1 | .12 |
| x2 | y1 | z2 | .18 |
| x2 | y2 | z1 | .20 |
| x2 | y2 | z2 | .10 |

**P(x1) = add every row with x1 (sum out Y, Z)**

P(x1) = .06 + .09 + .14 + .11 = **0.40**
P(z1) = .06 + .14 + .12 + .20 = 0.52
P(x1, z1) = .06 + .14 = 0.20

$$P(x1 \mid z1) = \frac{0.20}{0.52} = \mathbf{0.3846}$$

given part goes on the bottom

## Counting rows / models

**rows = $2^n$ (n = number of binary variables)**

$2^5$ = **32**  $2^7$ = **128**

each binary variable doubles it. 3 values each → $3^n$

# FILL-IN-THE-BLANK ONE-LINERS

## Formulas they blank out

Greedy f(n) = **h(n)**  UCS f(n) = **g(n)**  A* f(n) = **g(n) + h(n)**
BFS = **FIFO queue**  DFS = **LIFO stack**
IG = H(parent) − **weighted avg H(children)**  pure node H = **0**, 50/50 H = **1**
σ(0) = **0.5**  ReLU(neg) = **0**  tanh(0) = **0**
U(s) = R(s) + γ **$\max_a \Sigma P(s'|s,a) U(s')$** (Bellman)
VAE reparam: z = μ + **σ** · ε
P ⇒ Q ≡ **¬P ∨ Q**  P ⇔ Q ≡ **(P ⇒ Q) ∧ (Q ⇒ P)**
¬(A ∧ B) ≡ **¬A ∨ ¬B**  ¬(A ∨ B) ≡ **¬A ∧ ¬B**
P ⇒ Q ≡ **¬Q ⇒ ¬P** (contrapositive)
marginalise = **sum** over the other variables  joint rows = **$2^n$**
P(A|B) = **P(A ∧ B) / P(B)**  normalise: α = **1 / (s(b) + s(¬b))**
category reasoning: perceive → **categorise** → predict
the bold part is what usually gets blanked